

EchoVerse (Web Chat App) — Interview Q&A

Level: Beginner → Medium | **Questions:** 30 **Stack (resume):** WebRTC, Node.js, Socket.IO, Express.js **Repo:** <https://github.com/ermadhav/ChatApp>

Important: These answers are based on your README and resume, not your source code. Change any answer so it matches what you actually built. If you didn't implement something (TURN server, database, cooldowns), say so honestly and describe how you would add it. That answer is stronger than a bluff.

Check before the interview

- Your README lists React, MongoDB and Tailwind, but the repo is mostly plain JS/CSS/HTML (`app.js` + `public/`). Decide exactly what you used and answer Q5 accordingly.
- The repo has a `vercel.json`. Vercel serverless functions don't keep WebSocket connections alive, so know where your Socket.IO server actually runs (see Q28).

Table of contents

1. Basics
 2. Node, Express and Socket.IO
 3. WebRTC
 4. Features: codes, stranger mode, sharing, recording
 5. Security, deployment and scaling
 6. Reflection
-

1. Basics

Q1. Explain this project in 30 seconds.

A: EchoVerse is a real-time chat and video calling web app. You can connect privately with a friend using a personal code, or turn on Stranger Mode to be paired with a random person. It supports text chat, video calls, screen sharing, screen recording, and camera/mic toggles. Node.js and Express serve the app, Socket.IO handles real-time messaging and signaling, and WebRTC carries the video.

Q2. Why did you choose to build this?

A: I wanted to learn how real-time apps work beyond normal request/response: sockets, peer-to-peer media, and handling users joining and leaving. Video calling touches networking, browser APIs, and backend logic together, so it taught me a lot.

Q3. Which technologies did you use and why?

A:

- **Node.js + Express:** JavaScript on the server, and Express to serve the pages and set up the server.
- **Socket.IO:** real-time two-way events and the signaling channel.
- **WebRTC:** browser-native peer-to-peer audio/video.
- **HTML/CSS/JS frontend:** the UI for the call and chat.

Q4. What is the difference between HTTP and WebSocket?

A: HTTP is request/response: the client asks, the server answers, and the connection is done. A WebSocket stays open, so both sides can send messages at any time. That's what makes chat and live signaling possible without constantly polling.

Q5. Your README mentions React and MongoDB. Where did you use them?

A: (*Answer honestly.*) If you didn't use them: "The README's tech list was written broader than the final build. The working version uses Node, Express, Socket.IO and WebRTC. Users are identified by session codes held in server memory, so no database was needed." If you did use them, name what is stored (for example users or messages) and why.

Q6. What is the event loop in Node.js, and why does it suit this app?

A: Node runs JavaScript on a single thread and uses an event loop to handle many connections without blocking. A chat server mostly waits on I/O, so it can handle thousands of open sockets efficiently, as long as I don't run heavy CPU work on the main thread.

2. Node, Express and Socket.IO

Q7. What is Express, and what does it do in your project?

A: Express is a minimal web framework for Node. In this project it serves the static frontend from `public/` and creates the HTTP server that Socket.IO attaches to.

Q8. What is Socket.IO, and why not use plain WebSockets?

A: Socket.IO is a library built on WebSockets. It adds automatic reconnection, named events, rooms, acknowledgements, and a fallback to long-polling if WebSockets are blocked. Plain WebSockets would have meant writing all of that myself.

Q9. What is the difference between `socket.emit`, `socket.broadcast.emit` and `io.to(room).emit`?

A:

- `socket.emit`: sends to only that one client.
- `socket.broadcast.emit`: sends to everyone except that client.
- `io.to(room).emit`: sends to everyone in a specific room.

Q10. What are Socket.IO rooms?

A: A room is a named group of sockets. You can join a socket to a room and then send messages to just that group. It's a natural fit for a private one-to-one session.

Q11. What is "signaling", and why does WebRTC need it?

A: Two browsers can't connect directly until they know how to reach each other and what media formats each supports. Signaling is the process of exchanging that information (offers, answers, ICE candidates) through a server. WebRTC doesn't define how to do signaling, so I used Socket.IO.

Q12. How do you handle a user disconnecting?

A: Socket.IO fires a `disconnect` event on the server. In that handler I remove the user from the code-to-socket mapping and from the waiting queue, and notify their partner so the partner's UI can close the call and release the camera.

Q13. Two strangers click "connect" at the same instant. Could one get matched twice?

A: On a single Node process, code runs one event at a time, so checking the queue and pairing users happens in one uninterrupted step and there's no race. With several server instances this would become a real problem, and I'd need shared state like Redis with atomic operations.

3. WebRTC

Q14. What is WebRTC?

A: WebRTC is a set of browser APIs for real-time audio, video and data between peers, with no plugins. Media flows directly between browsers, which keeps latency low and server costs small.

Q15. Is the video going through your server?

A: No. Only the signaling messages go through my server. Once the connection is established, audio and video go directly peer-to-peer (unless a TURN relay is needed).

Q16. What is `RTCPeerConnection`?

A: It's the main WebRTC object. It manages the connection to a remote peer: adding media tracks, creating offers and answers, handling ICE candidates, and firing events when remote tracks arrive.

Q17. What are the offer, answer and SDP?

A: SDP (Session Description Protocol) is a text description of what a peer supports: codecs, media types, and so on. The caller creates an **offer** SDP, the callee replies with an **answer** SDP, and both apply them with `setLocalDescription` and `setRemoteDescription`.

```
const pc = new RTCPeerConnection({
  iceServers: [{ urls: "stun:stun.l.google.com:19302" }],
});
localStream.getTracks().forEach((t) => pc.addTrack(t, localStream));

const offer = await pc.createOffer();
await pc.setLocalDescription(offer);
socket.emit("offer", { to: partnerId, offer });
```

Q18. What are ICE candidates?

A: ICE (Interactive Connectivity Establishment) candidates are possible network paths between two peers: local IPs, public IPs found via STUN, or relay addresses. Each side sends its candidates through signaling, and WebRTC tests them to find one that works.

Q19. What are STUN and TURN servers?

A: A STUN server tells a device its public IP and port, which it needs behind a NAT or router. A TURN server relays the media when a direct connection is impossible, for example with strict corporate firewalls. STUN is cheap and often free. TURN uses real bandwidth. *(If you only used a public STUN, say so and add that production apps need TURN for reliability.)*

Q20. How do you get camera and mic access?

A: With `navigator.mediaDevices.getUserMedia({ video: true, audio: true })`. It returns a `MediaStream` that I attach to a `<video>` element as `srcObject`. It only works on HTTPS or localhost, and the user has to grant permission.

Q21. How do you show the remote user's video?

A: The `ontrack` event on the peer connection fires when the remote stream arrives. I take `event.streams[0]` and set it as `srcObject` of the remote `<video>` element.

Q22. What happens if the user denies camera permission?

A: `getUserMedia` rejects with an error like `NotAllowedError`. I catch it and show a message that the camera or mic is needed, instead of leaving a broken call screen.

4. Features

Q23. How does the personal-code feature work?

A: When a user connects, the server generates a unique code and stores a mapping from code to socket ID. When someone enters a friend's code, the server looks it up and forwards the connection request only to that socket. It keeps the connection private, and both people can choose video or chat.

Q24. How does Stranger Mode work?

A: Users who enable it go into a waiting queue on the server. If someone is already waiting, the server pairs the two and starts signaling between them. Otherwise the user waits. When a call ends or someone disconnects, both are removed from the pair and can be matched again.

Q25. How did you implement screen sharing?

A: With `navigator.mediaDevices.getDisplayMedia()`, which asks the user to pick a screen or window. Then I swap the outgoing video track using `sender.replaceTrack(screenTrack)` so the peer sees the screen without renegotiating the whole connection.

Q26. How did you implement screen recording?

A: I used the `MediaRecorder` API on a `MediaStream`. It emits chunks through `ondataavailable`, I collect them into an array, and on stop I combine them into a `Blob` and offer it as a downloadable file.

Q27. How do the camera and mic toggle buttons work?

A: Each media track has an `enabled` property. Setting `track.enabled = false` stops sending that media without closing the connection, and setting it back to `true` resumes it.

5. Security, deployment and scaling

Q28. Where did you deploy it, and are there any hosting limits?

A: *(Answer with what you actually did.)* Persistent WebSocket connections need a long-running server. Serverless platforms like Vercel functions don't keep them open, so the Socket.IO server normally runs on something like Render, Railway or a VPS. If your repo has a `vercel.json`, be ready to explain how the real-time part still works, or that it's a known limitation.

Q29. What security or safety concerns exist in a stranger-chat app?

A:

- **Abuse and inappropriate content:** I'd add report/block buttons, and rate limiting.
- **XSS in chat messages:** never insert user text as HTML. Use `textContent` or escape it.
- **HTTPS:** required for camera access and to protect signaling traffic.
- **Privacy:** peers can see each other's IP through ICE, which I'd mention in a privacy note.

Q30. How would you scale this to thousands of users?

A:

- Run several Socket.IO servers with the **Redis adapter** so they share events and state.
- Add a **TURN server** for reliable connections.

- For group calls, use an **SFU** (like mediasoup) instead of a full peer-to-peer mesh, which doesn't scale.
 - Keep the matching queue in Redis so pairing stays consistent across servers.
-

6. Reflection

Have short, honest answers ready for these follow-ups:

- **Biggest challenge?** Good choices: getting offer/answer/ICE order right, handling disconnects mid-call, or connection failures on different networks. Explain how you debugged it (for example `chrome://webrtc-internals`).
- **What would you improve?** TURN server, message history in a database, login, moderation tools, tests.
- **What did you learn?** How signaling works, how peer-to-peer differs from client-server, and how to design real-time state on the server.